WEEK 2 — DEVELOPERS

Ship features faster with an AI-paired workflow

أنجز الميزات أسرع مع سير عمل مقترن بالذكاء الاصطناعي

YOUR AI TOOL

Gemini

Tuned for Gemini — lead with the goal, attach context, and ask for a numbered plan you confirm before execution.

مهيّأة لـ Gemini — ابدأ بالهدف، أرفق السياق، واطلب خطة مرقّمة تأكّدها قبل التنفيذ.

YOUR SYSTEM · BASH

Linux

Commands use bash. Run inside a scoped, authorized environment only.

الأوامر بـ bash. شغّلها داخل بيئة مصرّح بها فقط.

The Ship-Velocity Kit: Pairing With an AI Assistant Without Slowing Down

هذا الدليل العملي يعلّمك كيف تنجز الميزات أسرع عندما تعمل جنبًا إلى جنب مع مساعد برمجي ذكي. الفكرة ليست أن تكتب كودًا أقل، بل أن تفكر بوضوح أكبر، وتراجع بصرامة، وتحوّل المساعد إلى زميل منضبط بدل مولّد عشوائي للنصوص.

Speed in software has never come from typing faster. It comes from making fewer wrong turns, catching mistakes early, and never re-solving a problem you already solved. An AI assistant amplifies whatever workflow you already have. If your process is sloppy, it produces sloppy code at high speed. If your process is disciplined, it becomes a force multiplier. This kit is the disciplined version: a set of repeatable moves that turn an AI assistant from a clever autocomplete into a genuine pairing partner you can trust with real work.

Frame the Task So the Model Can Actually Execute

The single biggest cause of bad AI output is a vague request. Models do not read your mind, and they do not know your codebase's unwritten rules. Before you ask for any code, spend two minutes assembling context.

A good task frame includes four things:

1. **The goal in one sentence** — what behavior changes from the user's point of view.
2. **The constraints** — language, framework version, libraries you must or must not use, performance limits.
3. **The surrounding shape** — the file structure, the function signatures it must fit between, the patterns already used nearby.
4. **The definition of done** — what a passing result looks like, including edge cases.

The reason this works: a model's first answer is anchored heavily on the prompt. Every fact you omit becomes a guess, and guesses compound. Front-loading context is cheaper than correcting three rounds of drift later.

Spec First, Then Plan, Then Code

Resist the urge to ask for a finished implementation in one shot. The fastest path is almost always a short two-step loop: agree on a plan, then build against it.

Ask the assistant to restate the problem and propose an approach *before* writing code. Read that plan critically. Plans are cheap to fix; code is expensive to fix. If the plan misunderstands the data flow or skips an edge case, you catch it in ten seconds of reading instead of after a full implementation.

```
PROMPT TEMPLATE — Plan before code

I want to implement: <one-sentence goal>.

Context:
- Language / framework: <name + version>
- This must fit into: <file or function it slots into>
- Constraints: <must use / must not use / perf or memory limits>

Do NOT write code yet. First give me:
1. A 3-6 step plan of how you'd implement this.
2. The edge cases and failure modes you'll handle.
```

```
3. Any assumption you're making that I should confirm.

Wait for my "go" before writing any code.
```

This gates execution behind a cheap review step and forces hidden assumptions into the open where you can correct them.

Test-First, With the Assistant Writing the Tests First

Tests are where AI pairing pays off most, because tests are the contract that catches the model's mistakes for you. Ask for the tests before the implementation, and you get two benefits: the behavior gets pinned down precisely, and you now have an automatic verifier for whatever code comes next.

Drive it like this: describe the function's contract, ask for a focused set of tests covering the happy path plus the nasty edges, review those tests yourself, then ask for the implementation that satisfies them. Run the tests. Red, then green, then move on.

```
PROMPT TEMPLATE — Tests first

Here is the contract for a function I need:
- Name & signature: <signature>
- It should: <behavior>
- Inputs / outputs: <types and shapes>
- Edge cases I care about: <empty input, nulls, large input, etc.>

Write ONLY the tests for now, using <test framework>.
Cover the happy path and each edge case as a separate, clearly named test.
Use Arrange-Act-Assert and descriptive test names.
Do not write the implementation yet.
```

The why: when the test suite is authored before the code, it cannot be quietly shaped to rubber-stamp a buggy implementation. The tests stay an independent check.

Review AI Output Like It Came From a Stranger

Never paste generated code straight into your branch. Treat every block as a pull request from a fast but overconfident junior. The model is excellent at producing plausible code and notoriously willing to invent APIs, swallow errors, and miss security checks.

Run this mental pass on every chunk:

- **Does it actually compile and run?** Don't trust it — execute it.
- **Are the libraries and functions real?** Hallucinated method names are common.
- **Is every error handled, or are failures silently swallowed?**
- **Any hardcoded secret, key, or unsanitized input?** These slip in constantly.
- **Does it match our existing patterns,** or did it invent a new style?
- **Is it doing more than I asked?** Extra "helpful" code is extra surface area for bugs.

The reason this matters: the model optimizes for looking correct, not for being correct. Your review is the only thing standing between plausible code and shipped code.

Refactor and Clean Up as a Deliberate Second Pass

First-draft AI code tends to be verbose, repetitive, and over-abstracted. Once tests are green, run a separate cleanup pass aimed only at clarity, never at behavior. Keeping the two passes separate is what makes the refactor safe — the tests prove you didn't change behavior.

Ask for specific, bounded improvements: collapse duplication, shorten functions over fifty lines, remove dead branches, replace magic numbers with named constants, and tighten naming. Avoid open-ended "make it better" requests, which invite the model to rewrite things you never asked it to touch.

```
PROMPT TEMPLATE — Cleanup pass

The tests below are passing. Refactor the implementation for clarity
WITHOUT changing behavior. The tests must still pass unchanged.

Specifically:
- Remove duplication and dead code.
- Split any function longer than ~50 lines.
- Replace magic numbers with named constants.
- Improve names so intent is obvious.
```

```
Do not add new features, new dependencies, or new abstractions.
Show only the changed code and explain each change in one line.
```

A Repeatable Daily Workflow

Velocity comes from a loop you can run on autopilot. Each task, regardless of size, runs through the same gates:

1. **Frame** — write the one-sentence goal, constraints, and definition of done.
2. **Plan** — get a short plan, read it, correct assumptions before any code.
3. **Test** — generate and review the tests that define success.
4. **Implement** — generate code against those tests; run them.
5. **Review** — read the output critically; verify APIs, errors, secrets.
6. **Refactor** — a separate clarity-only pass with tests still green.
7. **Commit** — small, focused commits with a clear message of what changed and why.

Keep tasks small. A model handles a tightly scoped task far better than a sprawling one, and small tasks mean small reviews and small blast radius when something breaks. When a request grows beyond a single clear outcome, split it. The discipline of small, well-framed units is what keeps the whole loop fast and trustworthy.

Checklist

- [ ] Every task starts with a one-sentence goal, explicit constraints, and a definition of done.
- [ ] Plan is reviewed and corrected *before* any code is written.
- [ ] Tests are authored and read before the implementation that satisfies them.
- [ ] All generated code is executed, not assumed to work.
- [ ] APIs and library calls are verified as real, not hallucinated.
- [ ] No swallowed errors, hardcoded secrets, or unsanitized input slip through review.
- [ ] Refactoring is a separate pass that keeps tests green and changes no behavior.
- [ ] Tasks are kept small enough for a small review and a small blast radius.
- [ ] Commits are focused, with a message stating what changed and why.